2016 12th International ACM SIGSOFT Conference on Quality of Software Architectures

Modeling the Failure Pathology of Software Components

João M. Franco, Frederico Cerveira, Raul Barbosa, Mário Zenha-Rela
CISUC, Department of Informatics Engineering
University of Coimbra
P-3030 290, Coimbra, Portugal
Phone: +351 239 790 000 Fax: +351 239 701 266
Email: {jmfranco, fmduarte, rbarbosa, mzrela}@dei.uc.pt

***Abstract*—Reliability prediction methods support early decision on software development by offering means for architects to reason and test their own designs. However, most of today's reliability prediction methods model the failure behavior as two states: functioning properly or failed, neglecting other failure modes. In addition, the lack of realistic case-studies and actual failure data make testing and validation difficult tasks.**

With this in mind, we propose a reliability modeling approach describing the behavior of a software component taking into account error occurrences and their associated impacts in terms of error masking, error propagation, error recovery and failures modes. To validate our approach we present a real case-study based on Xen where we injected faults at different virtualization layers. The experiments show close results between our modeling approach and the realistic values obtained from the experiment.

***Index Terms*—Reliability, Software architecture, Stochastic Modeling, Error, Failure, XEN**

I. Introduction

In past decades several methods [1]–[4] have been proposed to assess, predict and analyze reliability from software architectures. These methods promote early identification and correction of reliability issues early in the software development life-cycle and also, support evolutive changes in the architecture by providing analysis capabilities.

However, these methods present limitations due to their binary failure mode that only models correct and failure states [5]. In more detail, these methods assume that when an error occurs it manifests itself as an application failure, propagating to the application output. This assumption means that if an error occurs in one of the components, the whole application fails without taking into account the likely possibility of masking, recovery, or tolerate that error.

To address this limitation, we propose: 1) a method to increase the level of detail in traditional reliability modeling approaches, focusing on the software component, and 2) a practical experience that collects real reliability data to validate the proposed model.

In the first contribution, we model reliability through a stochastic model expressing the failure mode presented by Avizienis *et al.* [6]. The model outlines the failure process of a software component accounting for errors, failure recoveries, propagation, error masking and different failure modes.

Regarding the second contribution, the practical experiment uses fault injection into a running system on a widely used virtualization solution for cloud-based services – Xen. Xen supports the execution of multiple Operating Systems (OS) on the same hardware allowing us to have a control group to identify if the fault injected in a particular OS is propagated to another. In addition, Xen is distributed as an open-source software providing us the means to inject faults directly to the hypervisor by modifying its kernel extensions. The results allow to validate our approach with a realistic case-study showing that it achieves more accurate reliability predictions than classical studies. Moreover, the experimental testbed and its failure data serve as a contribution to the research community, since they can be used by other research studies to test, validate and benchmark different reliability methods.

This paper is organized as follows. Section II describes the background and related work. Section III introduces the proposed stochastic model to assess reliability from a software component and Section IV outlines the case-study of an actual cloud-based system. In Section V we discuss the experimental results obtained showing how they validate the proposed model, while Section VI concludes the paper.

II. Related Work

Software architecture provides a high-level of abstraction that allows designers to reason about the system and its topology through components and their interconnections [4]. For this reason, software architecture has been used as a source to model reliability through stochastic models [3], [7]–[9]. Cheung [2] was one of the firsts to propose reliability prediction from an architecture by using Deterministic-Time Markov Chains (DTMCs) and several surveys were presented since then [1], [10], [11].

These surveys overview the state-of-the-art pointing out venues for future work and current limitations. From the outlined limitations, Gokhale [10] reminds the importance of defining component failure models and identifies the area of parameter estimation techniques as being in the infancy at

978-1-5090-2567-1/16 $31.00 © 2016 IEEE
DOI 10.1109/QoSA.2016.15

that time (2007). Immonen and Niemelä [11] point out the lack of support for tools, weak reliability analysis of software components, and weak validation of the methods and their results. Our work addresses the above limitations by proposing a reliability prediction model that defines component failures models as suggested by Gokhale. In addition, we propose a practical experiment which addresses the lack of parameter estimation techniques and also serves as a validation of our modeling method addressing the limitation raised by Immonen and Niemelä.

Goseva-Popstojanova *et al.* [8] presented a comparison between different architecture-based reliability prediction methods based on an empirical, large scale and real case-study. Their work reveals that the literature assumes a too simplistic relationship between faults and failures, leading to errors in the analysis. This seminal paper concludes that more work should be conducted by the software testing and reliability research communities in order to explore more realistic relationships between faults and failures. This was a strong motivator for the work presented in this paper.

Cortellessa and Grassi [12] proposed a method to predict the system reliability by encompassing error propagation. In their work, authors recognize that they faced the same difficulties as other studies, namely the absence of reference values and a lack of parameter estimation techniques to obtain meaningful probabilities of actual internal failures and error propagation.

Filieri *et al.* [5] present a novel approach that deals with multiple failure modes and error propagation among components. Their approach supports the specification of individual components' attitude to produce, propagate, transform and mask different failure modes. Again, as pointed out by the authors in the concluding remarks, they are unable to assess their approach effectiveness due to the lack of real case-studies and actual values of reliability.

The proposed reliability model also accounts for error propagation which consists of an error occurring somewhere in the system. This error can be propagated to other components or to the application output. Hiller *et al.* [13] introduced the concept of error permeability and presented a framework to analyze the propagation and severity of data errors in a software system.

To assess the error propagation between components, Abdelmoez *et al.* [14] proposed an analytical method using fault-injection techniques to estimate the probability of error propagation in a software architecture. Johansson and Suri [15] also used fault-injection techniques to assess error propagation in an Operating System (OS). The OS is assumed to be a black-box and it is profiled through a fault injection technique and data error propagation analysis. The results showed that many errors do not propagate or propagate in a robust manner.

In common with some of the above error propagation studies, our approach relies on fault injection techniques to estimate the system failure data. However, it differs in the fact that we use the collected data to study the reliability impact and validate the proposed modeling approach. Thus, our goal is not only the estimation of the failure data of a system, but also provide a realistic, freely-accessible case-study, and actual failure data which can be used by researchers and practitioners to compare and validate different reliability prediction methods.

In short, this paper proposes a novel modeling approach to estimate reliability from a software architecture. It addresses the shortcomings highlighted by research surveys by proposing a more detailed failure behavior including masking, error propagation, multiple failure modes and recoveries. As a result, we validate our approach through a realistic case-study by collecting actual failure data and by asserting the adequacy of the proposed stochastic model.

III. Approach

Reliability prediction from a software architecture description has been largely studied and several methods have been presented [1], [16]. These methods model reliability through Continuous- or Discrete-Time Markov Chains (CTMCs or DTMCs). In short, the former implies the notion of time where reliability is denoted by the time elapsed since the start of the execution until a failure at time t occurs. The latter, defines reliability as a probability of delivering the correct service when it is requested for use.

Our approach extends the classical reliability modeling study of Cheung [2] by presenting a failure model more descriptive than the ones that encompass only two states: functioning properly and failure. We use DMTCs to estimate the reliability from a software architecture as Cheung proposes, but we extend them by encompassing errors, error masking, propagation to another components, different failure modes and failure recoveries.

A DTMC is a tuple $M_{\mathrm{D}} = (S, \overline{s}, P, L)$, where:

- S is a finite set of states;
- $\overline{s} \in S$ is the initial state;
- $P\colon S \times S \to [0,1]$ is the transition probability matrix where $\Sigma_{s' \in S} \mathbf{P}(s, s') = 1$ for all $s \in S$;
- $L\colon S \to 2^{AP}$ is a labeling function which assigns to each state $s \in S$ the set $L(s)$ of atomic propositions that are valid in the state.

The control is transferred from a state s_i to state s_j through a transition probability matrix which assigns to each pair of states (s_i, s_j) a probability $P(s_i, s_j)$. A transition only occurs if the probability of transiting between states i and j is greater than zero: $P(s_i, s_j) > 0$. If there is more than one transition from a state s_i, like $P(s_i, s_j) > 0$ and $P(s_i, s_k) > 0$, their sum must total to 1, $\sum_{j=1}^{k} P(s_i, s_j) = 1$.

Our approach models the failure process of a component as proposed by the dependability community and presented by Avizienis *et al.* [6]. This model encompasses errors, failures, recoveries, error propagation and different failure modes. To illustrate such model, Figure 1 presents a DTMC representing the failure process of a single software component.

In more detail, a software component represents a unit of computation and can be in either one of the eight states depicted in the modeled DTMC:

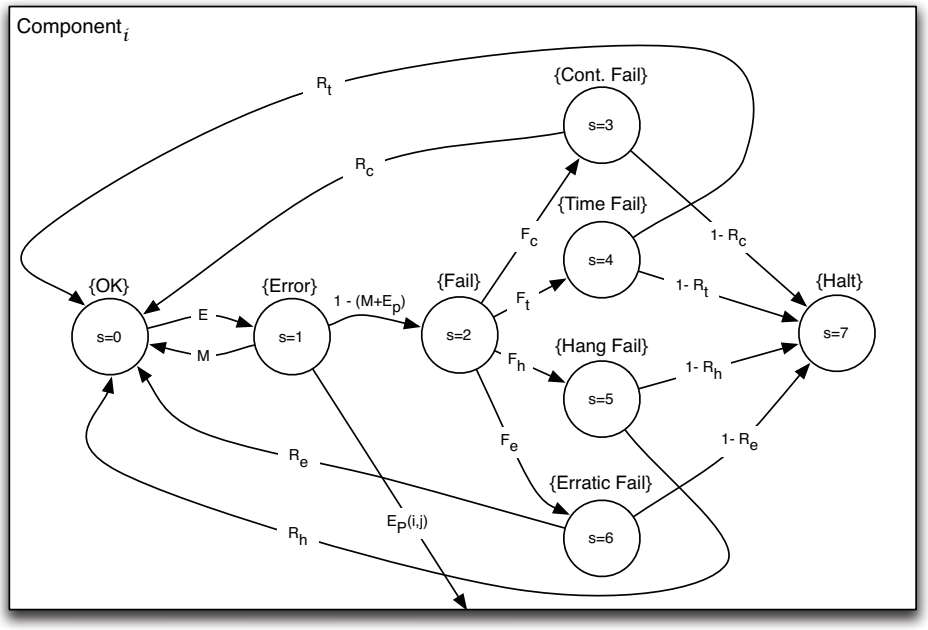

Fig. 1: DTMC illustrating the failure process of a single software component

- OK – the component is not in the presence of an error and executes as expected. When an error occurs with probability E the control of execution exits the OK state ($s = 0$) and enters in the error state ($s = 1$).

- Error – represents when a fault is activated leading the component to enter in an error state. When an error occurs, one of the following possibilities can happen:
  - the error is masked with probability M returning to the OK state ($s = 0$). An error can be masked through fault tolerance mechanisms, including error detection or recovery techniques which bring the system to a correct state;
  - it can enter in a failed state $(s = 2)$;
  - the error can be propagated to component j with rate $Ep_{(i,j)}$.

- Fail – this state ($s = 2$) denotes when an error is activated, leading to a failure. When a component fails, it can be categorized as content, timing, hang or erratic [6] with rate probability F_c, F_t, F_h, F_e, respectively.

- Content, Timing, Hang and Erratic Failure states – describe the multiple failure modes that follow into the categories of the failures proposed by Avizienis *et al.* [6]. After entering in one of these failure modes, the system can recover due to, as an example, the implementation of failure handling techniques. The recovering rates from the failure states content ($s = 3$), timing ($s = 4$), hang ($s = 5$) and erratic ($s = 6$) to the state OK ($s = 0$) are given by R_c, R_t, R_h, R_e, respectively.

- Halt – this state ($s = 7$) illustrates the complete stop of the system and emulates a failure that leads the system to a crash without the possibility of being recovered without human intervention.

To model the failure mode of a system, we use an absorbing DTMC which encompasses two final states with self-loop transitions defining the successful (S_c) and failure states (s_f). The rationale behind this approach is that each task is processed through the various states in the model and reaches one of two conditions: a successful (s_c) or a failure state (s_f).

To obtain a probabilistic quantification of the modeled system reliability we specify a property in the Probabilistic Computation Tree Logic (PCTL) [17]. This property computes the probabilities from the initial state $\overline{s}$ until the successful absorbing state ($state = s_c$) has been reached.

To model the interactions of different software components, several studies adopt a user-oriented approach [1], [2] which assumes a sequential order in the execution. However, a system may encompass different architectural patterns in which components may for example execute in parallel or concurrently. To model such behaviors, Wang *et al.* [18] propose a method to solve a DTMC according to each applied pattern. In this study we also assume a sequential order and other styles are addressed as future work.

The above modeling approach encompasses errors, their masking and propagation, failure occurrence, their recoveries

and different failure modes. As a result, it provides a complete model of the failure behavior of each component, allowing to produce more realistic and accurate models of a software system than classical reliability studies.

IV. Case-study

To validate the accuracy of our model we implemented a realistic case-study to compare the predicted reliability results with the ones obtained from real-world experiments. The case-study is based on a cloud infrastructure which deploys a set of HTTP servers responding to client requests. To ensure that our experiments were reproducible we adopted a widely known, freely available virtualization solution, the Xen hypervisor [19]. In practical terms, Xen is used in real world systems like the Amazon Elastic Compute Cloud (EC2), RackSpace Mosso or the CloudEx [20].

Figure 2 illustrates our case-study where Xen is depicted above the hardware layer and under the instanced virtual machines (VMs). The Dom0 is a privileged VM responsible for managing the virtualization environment and having direct access to the hardware. In this case-study we use two guest

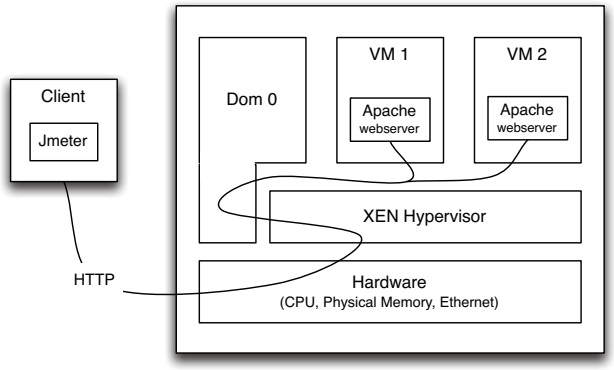

Fig. 2: Architecture of our case-study

virtual machines (VM1 and VM2) responsible for responding to client HTTP requests. Both VMs have the same system image and the same workload. Our goal is to inject faults in one of the VMs and monitor if the errors are propagated to the other one, i.e. if the isolation between both VMs gets compromised.

The edge depicted in Figure 2 from the Client to the VMs passing through the different components, illustrates the connection used for communication between client and servers. In addition, the format of the edge outlines the system dependencies among architectural elements. The client sends a request to a physical separated machine, where the case-study resides. This machine receives the request through its ethernet card (hardware), it is interpreted by the Xen hypervisor which is responsible to redirect the requests to the appropriate VM. Through the whole process, Dom0 is able to access the hardware and possibly monitor or modify this interconnection.

In this case-study the client is an external independent computation resource which performs HTTP requests concurrently to each VM through a Jmeter application. Each request is processed by the Apache webserver which is responsible for generating a file with a fixed size (1024MB) and content (zeroed file). Then, the webserver calculates a SHA1 hash of the generated file, resulting invariably in the same hash result unless an error occurs. This hash is then reported in the HTML response which can be assessed by the client whether it is the expected result or conclude that a failure has occurred. In this experiment we defined the size of the generated file to be 1024 MB. The reason for this figure is that small file sizes would result in a short life span for processing the response which might limit the effectiveness of an introduced error. As such, we opted for a large file size in order to increase the processing time to exercise CPU and memory, thus allowing the introduced error to be propagated and become noticeable.

This experiment includes a workload to simulate several HTTP clients performing requests to the available webservers. Thus, we configured Jmeter to use 10 clients that start by making requests during a 30 second ramp-up period and then to perform a request at each 600 milliseconds with 100 millisecond deviation pause between them. As a result, each experiment run has the duration of approximately 420 seconds (7 min) of which 330 seconds (5.5 min) concern the time where the workload is executed, and the remaining 110 seconds are divided among the setup period before the experiment (*i.e.*, launch probes and prepare the fault injection tool) and the cleanup period at the end (*i.e.*, extract logs, clean temporary files and restart machines).

A. Fault Injection

Estimating error propagation and failure occurrence probabilities can be a quite difficult task due to the number of factors involved and to the inherent randomness of operational profiles, inputs, failure types, etc. Thus, we use fault injection to artificially generate errors in the system as presented in the study of Cerveira *et al.* [21]. The goal is to collect relevant failure data and identify elements that are more prone to failure due to error propagation.

A fault is injected during the execution of the workload in an interval between 30 seconds and 4 minutes randomly chosen following a uniform distribution. This interval guarantees that the fault is injected after the ramp-up period and that the target system is working at its nominal throughput. It also provides enough time before the end of the experiment for the fault to be propagated.

The injected faults emulate transient hardware faults, by flipping bits in one of the available registers. Both the bit to flip and the register to target are chosen randomly following a uniform distribution.

B. Failure Classification

To model multiple failure modes we require the identification of the different type of failures that an error can lead to. With this in mind, we adapted the failure classification from Avizienis *et al.* [6] to our case-study as follow:

- Content – a failure occurs when the content of the received information deviates from the expected, correct one. Since in our case-study, for each request we generate a file with always the same size and content, its hash will invariably be the same. As such, whenever we receive a response with a different hash, we conclude that a content failure has occurred.

- Timing – occurs when the amount of time between sending a request and waiting for a response falls outside of a reasonable interval, leading to an early or late service failure. In this case-study, we assume that if a request is not responded between 2 and 15.78 seconds, it is considered as a timing-failure. These values have been extracted from the actual testbed after a large number of runs without fault injections. The maximum and minimum recorded response times were defined to be the upper and lower bounds of the acceptable time interval.

- Halt – this failure mode arises when there is an unexpected absence of system activity. In our experimentation, we account for a halt-type failure when a connection is dropped, suddenly closed or the server stops responding.

- Erratic – when the service is not halted but suffers a disruption in the delivered service, such as an arbitrary correct or incorrect response. In our case-study, we can assume that a failure is erratic if both the response content and timing deviates from the expected ones or the connection is still alive, but experiencing anomalies.

V. Experimentation

To prove the validity of the model proposed in Section III, we conducted a realistic experimentation to compare both the modeled and the realistic reliability values. In this section we present the gathered results[1], examine the validity of our approach and discuss the experimentation results.

A. Gather Failure Data

To validate our approach, we collected failure data by injecting faults into the system. In particular, we injected through two means: directly into the VM1 and in the Dom0. The goal of the former is to obtain relevant failure information about the impact of a transient hardware fault into the virtualized system. The injected fault only targeted one of the VMs (*i.e.*, VM1), leaving the other intact (*i.e.*, VM2). This aims to detect any breach in the isolation between both VMs which is usually assumed as guaranteed. The goal of the latter is to study the effect of introducing faults directly into the hypervisor.

1) Injection in VM1: The Xen hypervisor supports two types of virtualization: Paravirtualization (PV) where guest OSes run efficiently without requiring virtual emulated hardware and Hardware Virtual Machine (HVM) (also known as Full virtualization) in which guest OSes require the full emulation of hardware to run, such as BIOS, USB controllers or graphical adapters. The fault injection experiment in VM1 takes into consideration these types of virtualization by specifically making separate runs.

Table I presents the results for the experiment with the HVM virtualization type where we performed 1028 runs and injected a fault in each one. In those runs we observed 876 injected errors that have been masked and 152 that led to failure. Of those 152 failures, 4 manifested as content, 1 as timing, 147 as hang and 0 as erratic failures. Moreover, recovery was possible in every content and timing failure. However, in 22 runs occurred hang failures that could not be recovered, leaving the system in a complete inoperative state.

TABLE I: Hardware Virtual Machine (HVM) results

	HVM			
	VM 1		VM2	
	Failure	Recovery	Failure	Recovery
Content	4	4	0	0
Timing	1	1	0	0
Hang	147	125	0	0
Erratic	0	0	0	0
Total	152	130	0	0
Total Non-Failures	876		1028	
Total Halts	22		0	
Total Injections	1028			

Furthermore, the results show that from the 1028 faults injected in VM1, none has affected VM2. Thus, the experiment shows that the integrity of the isolation layer between virtual machines is preserved by flipping bits in CPU registers for the HVM virtualization type.

Regarding Paravirtualization (PV) results, Table II depicts the failure data obtained after 968 fault-injections. In those 968 runs we observed 876 errors that have been masked and 92 injections that led to failure. More specifically, 7 failures have been assigned as content type, 0 as timing, 85 as hang and 0 as erratic. The results show that all content failures have been successfully recovered, while 6 hang failures led the system to become unresponsive.

[1]The raw data of the experiments can be found at: https://eden.dei.uc.pt/~fmduarte/article_qosa_rawdata.tar.gz

TABLE II: Paravirtualization (PV) results

	PV			
	VM 1		VM2	
	Failure	Recovery	Failure	Recovery
Content	7	7	0	0
Timing	0	0	0	0
Hang	85	79	0	0
Erratic	0	0	0	0
Total	92	86	0	0
Total Non-Failures	876		968	
Total Halts	6		0	
Total Injections	968			

The results show that the injected faults in this virtualization type also did not affect the isolation layer between the Virtual Machines (VMs). This allows us to conclude that Xen is capable of confining error propagation between different guest VMs even when one VM is subject to fault-injection, in both virtualization types (PV and HVM)

2) Injection in Dom0: The privileged virtual machine Dom0 is responsible for managing the virtualization environment and has direct access to the hardware. Thus, we injected faults in this privileged VM aiming to analyze the system behavior and identify error propagation to the hosted VMs. To this end, we injected faults into the following kernel extensions:

- Qemu-system-i386: used by Xen to enable the dom0 to access a virtual disk;
- Xenwatchdogd: allows actions to be triggered when certain guest VMs are detected as crashed.

Table III presents the results of injecting faults into the Qemu kernel extension. We flipped bits in 101 runs in which both guest VMs have been affected. In those 101 runs VM1 and VM2 masked 94 errors while 7 led to failures causing the system to halt. Only one type of failure has been registered (hang) and the guest VMs were affected in the same way.

Table IV details the failure data of the injection in Xen-Watchdog extension. It can be observed that after the 73 injection runs, 56 were masked errors in both VMs. After the 17 triggered failures, none could be recovered. As in the previous experiment with the Qemu extension, only one type of failure has manifested, the hang-type failure, and again the injected faults affect equally both guest VMs.

In short, the results show that errors can be propagated from Dom0 to guest Virtual Machines, leading to a failure of a single type, the hang failure. It was also observed that when a failure occurs it cannot recover leading the system to a complete inoperative state. The observation that both guest VMs always display equal behavior can be explained by the fact that they share the same resources, system image, workload and are being subjected to the same fault-load.

TABLE III: Fault Injection in the Dom0 in the Qemu extension

	Qemu			
	VM 1		VM 2	
	Failure	Recovery	Failure	Recovery
Content	0	0	0	0
Timing	0	0	0	0
Hang	7	0	7	0
Erratic	0	0	0	0
Total	7	0	7	0
Total Non-Failures	94		94	
Total Halts	7		7	
Total Injections	101			

TABLE IV: Fault Injection in the Dom0 in the XenWatchdogd extension

	XenWatchdogd			
	VM 1		VM 2	
	Failure	Recovery	Failure	Recovery
Content	0	0	0	0
Timing	0	0	0	0
Hang	17	0	17	0
Erratic	0	0	0	0
Total	17	0	17	0
Total Non-Failures	56		56	
Total Halts	17		17	
Total Injections	73			

B. Validation

To validate the reliability model we propose two means: validate the model for each software component and then, confirm its accuracy regarding error propagation.

1) Single component validation: In this experiment we apply the values collected from the fault injection to the VM1 while taking into consideration both virtualization types. Our goal is to validate the proposed model and determine which virtualization type is more reliable. Table V depicts the failure data collected from the experiment results presented in Tables I and II.

To confirm the accuracy of the proposed model we compare its quantitative prediction with the real obtained values considering the following metrics: reliability, probability of entering in a failed state and the probability of each failure type.

The reliability results are presented in Table VI. These results show similar values between the real and the modeled one, bound by a 0.04% error margin. It can also be observed that the modeling achieved a better accuracy for the Paravirtualization (PV) than for the Hardware Virtual Machine (HVM).

TABLE V: Modeling Parameters

	HVM	PV
	VM 1	VM 1
M	0.8522	0.9050
F_c	0.0263	0.0761
F_t	0.0065	0.0
F_h	0.9671	0.9239
F_e	0.0	0.0
R_c	1.0	1.0
R_t	1.0	0.0
R_h	0.8503	0.9294
R_e	0	0

TABLE VI: Reliability

	Real	Modeled	Diff
HVM	97.86%	97.90%	0.04%
PV	99.38%	99.38%	0.0%

The probability of entering in a failed state is defined as an error being activated and leading to a failure. Table VII presents the reliability figures obtained from actual execution and values that resulted from modeling the system. The results depicted show a difference between both approaches of less than two percent. Regarding virtualization types, the PV presents better results since it has a lower probability of entering in a failed state.

TABLE VII: Probability of entering in a failed state

	Real	Modeled	Diff
HVM	14.78%	12.88%	1.9%
PV	9.50%	8.67%	0.83%

The last metric to validate our proposed model is the probability of occurrence of each failure type. Table VIII illustrates the outcomes from the actual experimentation and from our model considering the two virtualization types, HVM and PV. The results differ from 0.1% to 1.78% between the real and the modeled values, but in the overall they achieve similar probabilities. As in previous experiments, the PV virtualization type achieves better results than the HVM.

2) Error propagation validation: In this section we detail the validation of the proposed modeling approach taking into account error propagation. Figure 3 illustrates a Discrete-Time Markov Chain (DTMC) which was simplified to highlight the relevant states and omit the ones that do not have any transition to, such as the content failure state (F_c). Our approach models the client performing requests to the VM1. These requests pass through the Dom0 which may raise an error that can be propagated to other VMs. When an error is propagated from the Dom0 to one of the VMs it leads to only one type of failure, the hang. Since the occurrence of a hang failure brings

TABLE VIII: Probability of entering in each failure mode

		Real	Modeled	Diff
HVM	Content	3.89%	3.79%	0.10%
	Timing	0.09%	0.09%	0.00%
	Hang	14.29%	12.51%	1.78%
	Erratic	0%	0%	0%
PV	Content	0.72%	0.71%	0.01%
	Timing	0%	0%	0%
	Hang	8.78%	8.07%	0.71%
	Erratic	0%	0%	0%

the whole system to an inoperative condition, we modeled the transition from the halt state to the complete failure state. Moreover, the interaction between components Dom0 and VM1 is modeled through a sequential order, meaning that Dom0 executes and then passes the control to VM1.

Table IX depicts the failure data collected from Tables III and IV.

TABLE IX: Modeling Parameters for the Dom0 experiment

	Qemu			Xenwatchdogd		
	VM 1	VM 2	Dom0	VM 1	VM 2	Dom0
M	0.0	0.0	0.93	0.0	0.0	0.76
F_c	0.0	0.0	0.0	0.0	0.0	0.0
F_t	0.0	0.0	0.0	0.0	0.0	0.0
F_h	1.0	1.0	0.0	1.0	1.0	0.0
F_c	0.0	0.0	0.0	0.0	0.0	0.0
R_c	0.0	0.0	0.0	0.0	0.0	0.0
R_t	0.0	0.0	0.0	0.0	0.0	0.0
R_h	0.0	0.0	0.0	0.0	0.0	0.0
R_e	0.0	0.0	0.0	0.0	0.0	0.0
$E_{p(\text{dom0, VM1})}$	-	-	0.035	-	-	0.12
$E_{p(\text{dom0, VM2})}$	-	-	0.035	-	-	0.12

The results of this experiment and the estimations of our model are illustrated in Table X. The reliability values are almost identical in both approaches, with WatchDogD presenting the higher deviation between real and modeled results, a difference below 4%.

TABLE X: Dom0 Reliability as the probability of non-failure

	Real	Modeled	Diff
Qemu	93.06%	93.51%	0.45%
WatchDogD	76.71%	80.64%	3.93%

C. Comparison with Classical Methods

Classical reliability prediction methods ignore error masking, different failure modes, recoveries and error propagation to another components. These methods assume that when an error occurs, it immediately propagates as an application error.

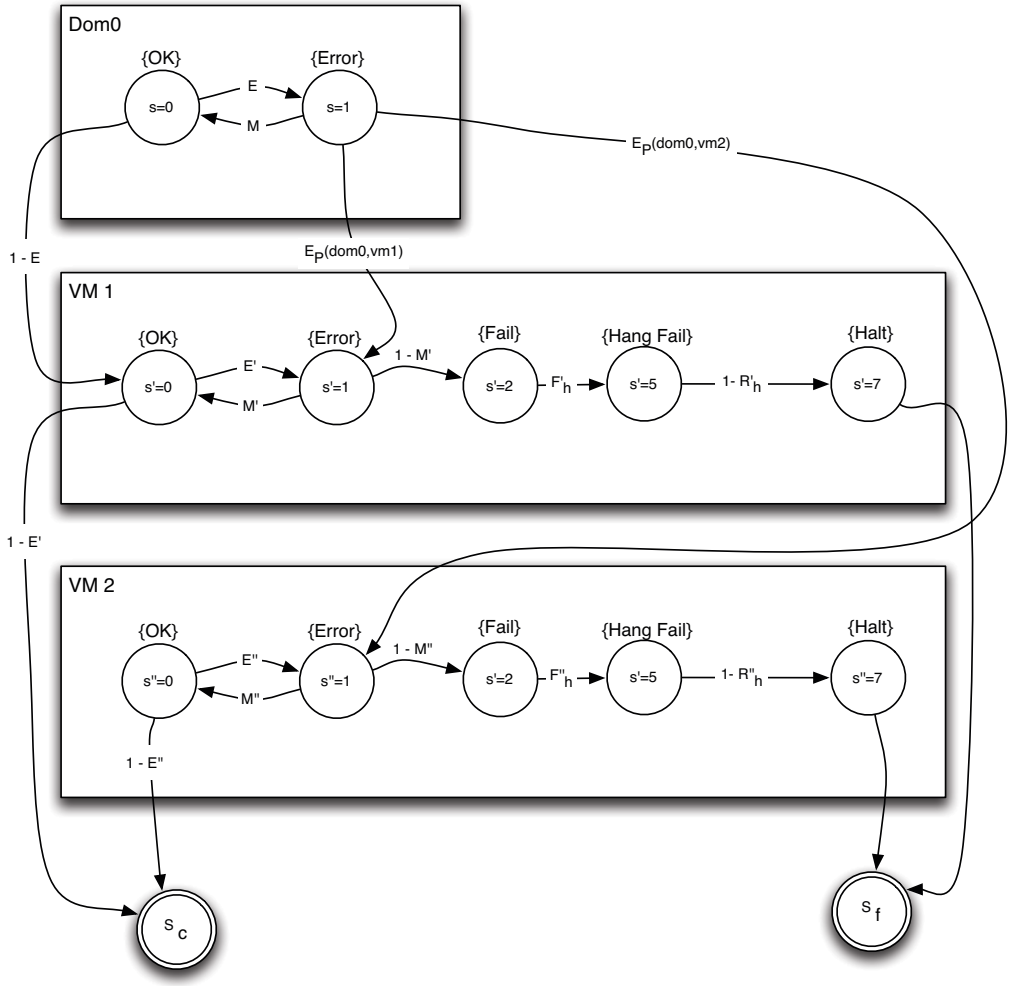

Fig. 3: DTMC modeling the Dom0 Fault Injection scenario

As a short example of the accuracy loss from classical methods, we resort to the results presented in Table I. In short, 1028 faults were injected into the VM1, resulting in 152 failures ($\approx 14.78\%$). This means that the total reliability for the VM1 component without taking into account recoveries, failure modes and propagation to another components would be of 85.22% which is 12.64% less than the real obtained reliability and 12.68% less than the modeled one, as presented in Table VI. In short, classical reliability models can be unrealistic and less accurate than our proposed model, specially in systems that employ failure recovery mechanisms.

D. Discussion

The gathered failure data allow us to conclude that the Paravirtualization (PV) type presents better results than the Hardware Virtual Machine (HVM). In addition, the HVM experienced timing failures while PV did not. We assume that this difference is related to the nature of the virtualization type. HVM implements a full emulation of the hardware which results in a slower performance than the Paravirtualization (PV) type, explaining the occurrence of timing failures in the HVM.

Furthermore, our experimental results show that the isolation layer between VMs is effective when a fault is injected in one of the VMs. However, when an error is injected in a kernel extension the error is not only propagated to the guest VMs, but it can also cause the system to fail and ultimately, lead it to a complete inoperative state.

Regarding the validity of our approach, both the estimations from our model and the values obtained from the actual Xen system are similar. The difference between both approaches fall in the interval $[0.0\%, 3.9\%]$, presenting in average a difference of 0.69% and a median of 0.07%. These figures support the validity our proposed modeling approach.

To conclude, we show that classical reliability models present different reliability values when recoveries, different failure modes and error propagation are taken into account.

VI. Conclusions

In this paper we propose a model to estimate the reliability of a software component that increases the level of detail from traditional reliability modeling approaches. The proposed model accounts with error masking, error propagation, failure occurrence, failure recoveries and multiple failure modes. In a practical sense, it supports early and evolutive assessment of software architecture by offering means for designers to reason and test and analyze their own designs in the presence of faults.

Additionally, we address the lack of realistic case-studies available in the research community to compare and validate newly proposed reliability prediction methods. To this end, we implemented a case-study based on a freely accessible widely used cloud infrastructure platform, the Xen Hypervisor.

To validate our proposal, we conducted a set of experiments with a comprehensive series of fault injection runs to emulate the presence of erroneous states in the system. The results show that the figures obtained from our proposed model are identical with the actual values obtained from the running system. In addition, we have been able to show that a cloud system may propagate errors from the hypervisor to the guest VMs, ultimately leading to their complete disruption. As a result, our approach was also able to accurately predict such behavior, thus validating our error propagation modeling.

As for future work, we intend to extend one of our previous work [22] to generate the proposed model in an automated manner from a software architecture. The goal is to ease the reliability estimation process by performing an automated estimation and analysis on the proposed design. In addition, we plan to extend the execution order of our DTMC to account with different styles like parallel or concurrent executions.

Acknowledgments

This research was supported by a a doctoral grant [SFRH/BD/89702/2012] and by the project DECAF: An Exploratory Study of Distributed Cloud Application Failures (reference EXPL/EEI-ESS/2542/2013), both from FCT – Fundação para a Ciência e a Tecnologia.

References

[1] K. Goseva-Popstojanova and K. S. Trivedi, "Architecture-based approach to reliability assessment of software systems," *Perform. Eval.*, vol. 45, no. 2-3, pp. 179–204, Jul. 2001.

[2] R. Cheung, "A user-oriented software reliability model," *IEEE Transactions on Software Engineering*, vol. 6, no. 2, pp. 118–125, 1980.

[3] R. Reussner, "Reliability prediction for component-based software architectures," *Journal of Systems and Software*, vol. 66, no. 3, pp. 241–252, Jun. 2003.

[6] A. Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr, "Basic concepts and taxonomy of dependable and secure computing," *IEEE Transactions on Dependable and Secure Computing*, vol. 1, no. 1, pp. 11–33, Jan. 2004.

[4] J. C. Munson, "Software faults, software failures and software reliability modeling," *Information & Software Technology*, vol. 38, no. 11, pp. 687–699, 1996.

[5] A. Filieri, C. Ghezzi, V. Grassi, and R. Mirandola, "Reliability analysis of component-based systems with multiple failure modes," in *Component-Based Software Engineering, 13th International Symposium, CBSE 2010, Prague, Czech Republic, June 23-25, 2010. Proceedings*, 2010, pp. 1–20.

[7] F. Brosch, B. Buhnova, H. Koziolek, and R. Reussner, "Reliability prediction for fault-tolerant software architectures," in *Proceedings of the joint ACM SIGSOFT conferenceQoSA and ACM SIGSOFT symposiumISARCS on Quality of software architecturesQoSA and architecting critical systemsISARCS*. ACM, 2011, pp. 75–84.

[8] K. Goseva-Popstojanova, M. Hamill, and R. Perugupalli, "Large Empirical Case Study of Architecture-Based Software Reliability," in *16th IEEE International Symposium on Software Reliability Engineering (ISSRE'05)*. IEEE, 2005, pp. 43–52.

[9] S. Yacoub, B. Cukic, and H. Ammar, "Scenario-based reliability analysis of component-based software," *Proceedings 10th International Symposium on Software Reliability Engineering (Cat. No.PR00443)*, pp. 22–31, 2000.

[10] S. S. Gokhale, "Architecture-Based Software Reliability Analysis : Overview and Limitations," *IEEE Transactions On Dependable And Secure Computing*, vol. 4, no. 1, pp. 32–40, 2007.

[11] A. Immonen and E. Niemel, "Survey of reliability and availability prediction methods from the viewpoint of software architecture," *Software & Systems Modeling*, vol. 7, no. 1, pp. 49–65, 2008.

[12] V. Cortellessa and V. Grassi, "A modeling approach to analyze the impact of error propagation on reliability of component-based systems," in *Component-Based Software Engineering*, ser. Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2007, vol. 4608, pp. 140–156.

[13] M. Hiller, A. Jhumka, and N. Suri, "An approach for analysing the propagation of data errors in software," *Proceedings International Conference on Dependable Systems and Networks*, pp. 161–170, 2001.

[14] W. Abdelmoez, D. Nassar, M. Shereshevsky, N. Gradetsky, R. Gunnalan, H. Ammar, B. Yu, and A. Mili, "Error propagation in software architectures," in *Software Metrics, 2004. Proceedings. 10th International Symposium on*, Sept 2004, pp. 384–393.

[15] A. Johansson and N. Suri, "Error propagation profiling of operating systems," in *Dependable Systems and Networks, 2005. DSN 2005. Proceedings. International Conference on*, June 2005, pp. 86–95.

[16] S. Gokhale, "Analytical models for architecture-based software reliability prediction: A unification framework," *Reliability, IEEE Transactions on*, vol. 55, no. 4, pp. 578–590, 2006.

[17] M. Kwiatkowska, G. Norman, and D. Parker, "Stochastic model checking," *Formal Methods for Performance Evaluation*, pp. 220–270, 2007.

[18] W.-l. Wang, D. Pan, and M.-H. Chen, "Architecture-based software reliability modeling," *Journal of Systems and Software*, vol. 79, no. 1, pp. 132–146, Jan. 2006.

[19] P. Barham, B. Dragovic, K. Fraser, S. Hand, T. Harris, A. Ho, R. Neugebauer, I. Pratt, and A. Warfield, "Xen and the art of virtualization," *SIGOPS Oper. Syst. Rev.*, vol. 37, no. 5, pp. 164–177, Oct. 2003.

[20] L. Qian, Z. Luo, Y. Du, and L. Guo, "Cloud computing: An overview," in *Cloud Computing*, ser. Lecture Notes in Computer Science, M. Jaatun, G. Zhao, and C. Rong, Eds. Springer Berlin Heidelberg, 2009, vol. 5931, pp. 626–631.

[21] F. Cerveira, R. Barbosa, H. Madeira, and F. Araujo, "Recovery for virtualized cloud environments," in *Proceedings of the 11TH European Dependable Computing Conference (EDCC)*, September 2015.

[22] J. M. Franco, F. Correia, R. Barbosa, and M. Zenha-Rela, "Affidavit: Automated reliability prediction and analysis of software architectures," in *INForum 2013*. 5th Portuguese Symposium on Informatics, September 2013, pp. 54–65.